1.

- array A = 10 elements
- register $s1 has base address of A
  - code segment swaps A[4] and A[9]

```
lw $t0, 16[$s1]
lw $t1, 36[$s2]
sw $t0, 16[$s2]
sw $t1, 36[$s1]
```

2.

$s1 = A1B2C3D4
$s2 = 5A6B7C8D

```
sra $t1, $s1, 3; // shift-right-arithmetic
srl $t2, $t1, 1; // shift-right-logical
```

1st Intruction: sra $t1, $s1, 3

$s1 = A1B2C3D4 = 10100001101100101100001111010100
Arithmetic Shift >> 3

11110100001101100101100001111010

Convert Back to Hex
$t1 = F436587A

2nd Instruction
0 11110100001101100101100001111010

→ 0111 1010 0001 1011 0010 1100 0011 1101
$t2 = 7A1B2C3D

3. sw $s1, 4($zero) into memory address 4
sw $s2, 8($zero) into memory address
lw $s1, 6($zero) a word from memory address

Memory address 4 = A1B2C3D4 (Bytes: A1 B2 C3 D4)
Memory address 8 = 5A6B7C8D (Bytes: 5A 6B 7C 8D)

Loading from address 6:
Byte 6: C3
Byte 7: D4
Byte 8: 5A
Byte 9: 6B

The final content for $s1 will be C3D45A6B
3 total memory accesses

4.

add $t1, $s2, $s1	R type	register addressing
a. lw $t0, 4($t1)	I type	base / displacement
bne $t0, $s5, End	I type	PC relative addressing
addi $s1, $s1, 2	I type	Immediate
subi $s1, $s1, 1	I type	Immediate
j start	J type	Pseudo direct

b. lw $t0, 4($t1)
opcode 35 (dec) 8 49

Binary: (100011) (01000) (0100) (01001) = (100011)(01001) / (01000)(0100)

= 1000 1101 0010 1000 | 0000 0000 0000 0100
Hex: 8D 280004

c. bne $t0, $s5, End
5 8 21 ?

Binary 000101 01000 10101 zero
opcode source target

0001 0101 0001 0101 | 0000 0000 0000 0000

Hex: 15150000

5. The jump (j) instruction in MIPS uses pseudo- direct addressing, where the instruction provides a 26-bit target address. Since full address in MIPS is 32 bits, the 26-bit target is shifted left by 2 bits, forming a 28-bit address. This is then combined with the upper 4 bits of the current PC, giving a 32-bit effective jump address. The PC is updated to this new address, and execution continues from that location. this allows jumps within a limited memory range, but sufficiently covers most program code locations.